

Title: Kernels of SafeAI: Differentially Private Analysis of User-LLM Interactions on End Devices

Project coordinator: Tal Wagner

1. Abstract

Large Language Models (LLMs) are increasingly deployed on end-user devices, creating unprecedented opportunities to analyze usage patterns while raising critical privacy concerns. This proposal presents a novel method for analyzing user interactions with on-device LLMs through differentially private (DP) mechanisms that preserve individual privacy while enabling population-level insights (Fig. 1).

Our approach leverages Sparse Autoencoders (SAEs) pre-trained for mechanistic interpretability to extract meaningful feature representations from LLM intermediate activations during user interactions. These high-dimensional embeddings are collected and analyzed across distributed devices using shuffled DP, ensuring strong privacy guarantees, while leveraging a recently proposed shuffled DP kernel density estimation (KDE) framework to identify the most significant SAE features that characterize global interaction patterns.

The proposed research addresses the fundamental challenge of analyzing distributed human-LLM interaction patterns at scale while maintaining rigorous privacy guarantees, leveraging the power of foundational algorithmic advances

combined with state-of-the-art empirical AI methods. Expected outcomes include a robust, scalable privacy-preserving framework for data collection and analysis from LLM usage in decentralized settings. The significance of the proposed research extends to AI safety, user experience optimization, and privacy-respecting LLM-powered systems.

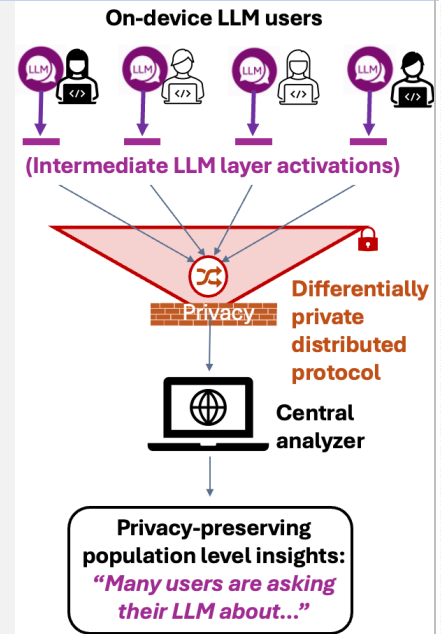


Figure 1: Overview of proposed method for privacy-preserving analysis of user-LLM interaction on end devices.

2. Scientific background and state of the art

2.1 On-Device Large Language Models

The deployment of LLMs on edge devices is a major goal and a growing trend in AI usability. Recent advances in model compression, quantization, and specialized hardware enable LLMs to run locally on personal computers and even smartphones and tablets [45-47]. This distributed deployment model offers reduced latency, enhanced privacy, and independence from cloud infrastructure, but creates new challenges for understanding collective usage patterns and model behavior. Current on-device LLM implementations focus primarily on inference optimization and resource efficiency, with limited attention to post-deployment analysis and understanding of interaction dynamics. Recent centralized privacy approaches have made valuable progress in user-LLM interaction analysis protecting user privacy [7-8]. However, they inherently require plain data sharing with central servers, which entails privacy vulnerabilities during data collection as well as significant data governance challenges. This gap creates opportunities for complementary approaches that can provide insights while keeping unprotected user data entirely on local devices. We note that we use the term “LLM” throughout, even though the size of the AI model (and whether it can be considered “large”) varies by use case and end device. Some language models considered “large” can run nowadays on personal computers, while smartphones are more limited. The proposed approach extends across model sizes.

2.2 Mechanistic Interpretability via Sparse Autoencoders (MI-SAEs)

Mechanistic interpretability seeks to understand the internal workings of neural networks by analyzing their learned representations and computational processes. SAEs have emerged as a powerful tool to this end [34-36]. Their premise is that dense intermediate layer activations can be decomposed into interpretable, sparse feature representations, and this decomposition can be recovered by training an SAE.

Recent work has demonstrated that SAEs can identify meaningful interpretable concepts within internal activations of transformer-based language models. These sparse features often correspond to interpretable semantic concepts, syntactic patterns, or reasoning processes, making them valuable for understanding user-model interaction. Many LLMs today have publicly available pre-trained SAEs with feature extraction and interpretation pipelines which are widely used in research and in applications [37-39]. See Fig. 2 for illustration and sec. 4.2.1 for details.

2.3 Differential Privacy (DP) in Distributed Data Analysis

Differential privacy provides mathematically rigorous guarantees for privacy protection in data analysis and is widely considered the current “gold standard” for privacy in ML and AI [9-10]. DP in distributed settings has been an active area of research due to important applications in end-device scenarios. The shuffled model of DP has recently risen to prominence in distributed ML settings, offering accurate, practical and privacy-preserving data collection and aggregation protocols by combining DP with secure computation techniques for anonymization [11-13]. See sec. 4.2.1 for a formal discussion.

Most work on shuffled DP in ML so far has focused either on traditional statistical queries (e.g., counting and summation) [20-22], or federated learning via iterative distributed optimization [23-27]. Recent work by the PI has made first steps in shuffled DP for distributed semantic analysis over high-dimensional dense vector embeddings using KDEs [1]. However, its use in LLM interpretability settings remains so far unexplored.

2.4 Differentially Private Kernel Density Estimation (DP KDE)

A *kernel* in this proposal is a measure of similarity between two elements from a pre-specified universe. Relevant examples to us are the inner product or the Gaussian kernel between vectors in $\mathbb{R}^d$, or the Jaccard similarity between sets. Kernel Density Estimation (KDE) is the average kernel similarity between a query element and each element in a dataset. It is a classical and widely used method in statistics and data science. Recently, differentially private KDE has been an active research area, particularly for high-dimensional vectors which are common in ML settings [1-6]. Recent work by the PI has developed a distributed DP KDE protocol with shuffled DP, which forms the basis for the method described this proposal, and demonstrated it can be applied to privacy-preserving semantic textual analysis [1].

2.5 Advantage of the Proposed Research

The main hypothesis of this proposal is that *shuffled DP KDE combined with SAEs can lead to AI interpretability insights under a strong notion of distributed privacy, improving the start-of-the-art for interpreting user-LLM interactions, particularly in the highly important emerging use case of on-device LLMs*. This hypothesis is strongly supported by PI’s preliminary research on shuffled DP KDE in a closely related semantic analysis context [1].

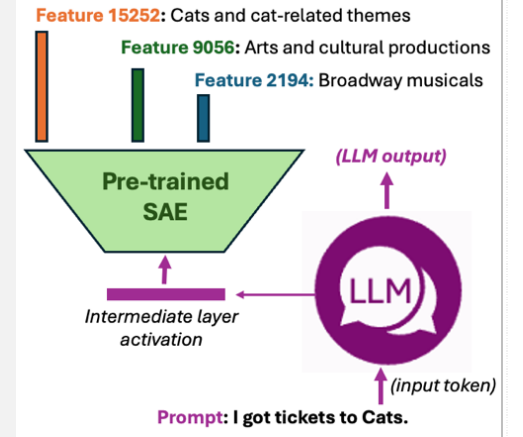


Figure 2: Illustration of SAEs in mechanistic interpretability of LLMs. Features shown are from Google DeepMind’s public pre-trained SAE for Gemma-2-2B, layer 24. The depicted example can be recreated with the Neuronpedia web interface ([link](#)).

Current approaches to analyzing human-LLM interaction are subject to a fundamental tension between the need for accurate insights and the imperative to protect user privacy. Existing solutions typically involve either complete data centralization (compromising privacy) or purely local analysis (limiting insights). No existing framework combines the interpretability benefits of SAEs with the privacy guarantees of DP for distributed LLM interaction analysis. This gap may limit the availability and deployment of tools for understanding collective AI usage patterns while respecting individual privacy rights, limiting our ability to improve AI systems and ensure their safe deployment.

3. Research objectives and specific aims

The primary objective is to develop and validate a comprehensive framework for privacy-preserving analysis of user interactions with on-device LLMs through differentially private aggregation and mechanistically interpretable features. Specific aims include:

Aim 1: Method design. Design a method for privacy-preserving aggregation and analysis of semantic representations of user interactions with on-device LLMs. Explore ways to extract and aggregate on-device LLM activations that enable interpretable insights while lending themselves to privacy-preserving distributed aggregation and analysis with appropriate choices of kernels for DP KDE protocols.

Aim 2: Privacy and utility analysis. Establish theoretical privacy guarantees for the proposed method and utility analysis to determine optimal privacy-utility trade-offs. Integrate adaptive privacy budgeting strategies for longitudinal analysis scenarios.

Aim 3: Prototype implementation and empirical evaluation. Implement the framework in controlled experimental settings with publicly available models and datasets. Validate the effectiveness of proposed method with comprehensive empirical analysis, ablations, and comparison to appropriate baseline methods.

4. Detailed description of the proposed research

4.1 Framework Overview

The proposed framework consists of four primary components operating in a distributed environment:

- **On-device processing:** Each participating device runs a local copy of the target LLM. During user interactions, the system captures intermediate layer activations from the LLM, which are to be processed and shared under a privacy-preserving aggregation protocol with the central analyzer.
- **Privacy-Preserving Aggregation Protocol:** The captured intermediate activations form distributed inputs to a shuffled DP KDE protocol coordinated by a central analyzer. The protocol entails local data protection combined with anonymized secure aggregation (as per the shuffled DP model).
- **Central Analyzer:** The protocol results in a privacy-preserving representation of the KDE function of the distributed user activation data. The central analyzer uses it for population-level analysis and insights.
- **Interpretable SAE feature extraction:** The target LLM is endowed with a pre-trained SAE to extract interpretable insights from intermediate layer activations of user-LLM interactions. The SAE features are used as queries to the privacy-preserving KDE function collected from the user data. The SAE may be invoked either on the end device or on the central analyzer, with different choices of kernels; see detailed discussion below.

4.2 Technical Outline

We now go into the system components and their associated algorithmic considerations in more detail.

4.2.1. Shuffled DP. We first describe shuffled DP in more technical detail. A shuffled DP protocol works as follows. First, each user locally adds some noise to their input and converts it into a stream of messages. Then, the users

participate in a secure aggregation mechanism to output a randomly permuted stream of all their messages. Secure aggregation can be performed with several well-established cryptographic techniques. Note that each step by itself is not enough: local noise obscures the content of the user input, but not the user’s identity; guaranteeing DP for a single identifiable user input (known as local DP) requires high noise levels (provably higher than central DP) that degrade accuracy. Cryptographically secure permutation of the messages ensures they can no longer be attributed to their original senders (effectively anonymizing them) but does nothing to obscure their content. It is the combination of the two steps that sets shuffled DP apart. Formally, the local noise needs to be designed such that when composed with the subsequent random permutation, the anonymized collection of messages satisfies a DP guarantee. Thus, the formal (ϵ, δ) -DP guarantee, $\Pr[\Pi(X) = v] \leq e^\epsilon \Pr[\Pi(X') = v] + \delta$, is required to hold with $\Pi(X)$ being the unordered stream of random messages generated by all users in a user database X , for all user database pairs X, X' that differ on a single user, and all supported messages streams v . Shuffled DP offers a strong notion of distributed privacy – the users never share their unprotected data with a central authority – while attaining levels of accuracy that can provably reach those of central DP [12-19].

4.2.2 Precursor work. For a start, we review the application of DP KDE to distributed textual semantic analysis from the PI’s precursor work [1]. It uses a pre-trained semantic embedding model which embeds texts as vectors in $\mathbb{R}^d$, such that semantically related texts have large inner product between their embeddings. Thus, the embedding of a textual concept like “movie” will attain a large KDE score over an embedded corpus of texts about movies.

The method in [1] has each end user embed their text and participate in a shuffled DP KDE protocol, at the end of which the central analyzer holds a privacy-preserving representation of the KDE function of the users’ text corpus. Since the KDE function representation is itself DP, by post-processing, it allows the server to test an unlimited number of KDE queries without further privacy leakage. It queries all elements in a publicly curated dictionary of keywords, and outputs the keywords with highest DP KDE score. See Fig. 3 for illustration.

For example, given a “private” corpus of Wikipedia snippets, most of which are about films, the method reports: “*movie, filmography, screenplay*”. Given a “private” corpus of news articles mostly about sports, it reports: “*injury, semifinalists, finalists*”. Importantly, this is done with shuffled rather than central DP – so these keywords are privately extracted *without the central server observing even a single unprotected text*, yielding a strong notion of distributed end-user privacy. This is achieved by the combined power of deep learning embedding models, KDE, and shuffled DP.

4.2.3. Adaptation to user-LLM interactions. The core contribution of the current proposal is to extend this approach to on-device user-LLM interactions, applying it to intermediate layer activations. This approach has notable natural advantages over the method in [1]: First, intermediate layer activations are already a byproduct of user-LLM interaction – they are computed anyway, and re-using them for additional population-level insight entails minimal computational overhead, whereas the framework in [1] necessitates embedding the texts with a pre-trained embedding model even if they aren’t otherwise embedded for another purpose. Second, the availability of pre-trained SAEs for

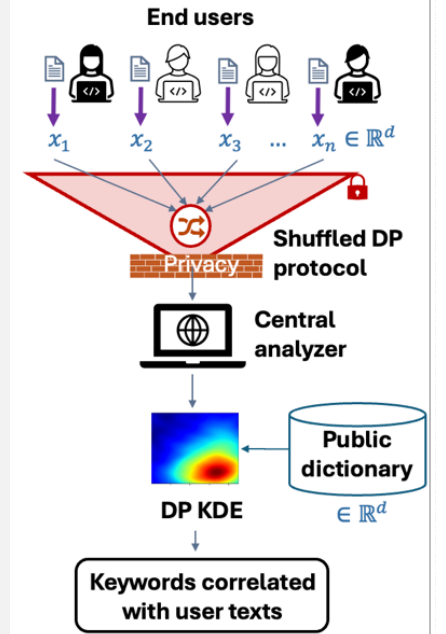


Figure 3: Illustration of DP KDE for private distributed semantic analysis, as proposed in [1].

intermediate activations obviates the need for curating a public dictionary of keywords to form KDE queries – the pre-trained SAE features naturally serve as KDE queries.

On the other hand, there are considerable challenges. Central to them is the fact that while the text embedding models used in [1] are pre-trained directly for semantic interpretability, intermediate LLM layer activations are a byproduct of their end-to-end training process and are not interpretable “out-of-the-box”. This is where *mechanistic interpretability* comes into play as a paradigm aiming to endow intermediate activations with interpretable explanations.

4.2.4. Interpretability with MI-SAEs. To better understand the interplay of MI-SAEs with our proposed approach, we describe them in more detail. Let d be the dimension of intermediate LLM activations in a chosen layer. The goal of training the SAE is to discover N “features” $\varphi_1, \dots, \varphi_N \in \mathbb{R}^d$, where $N \gg d$ is a pre-specified parameter, such that every input activation $x \in \mathbb{R}^d$ admits an approximate sparse representation $x \approx \sum_{i=1}^N \alpha_i \varphi_i$ where only a small number of coefficients α_i are non-zero. The corresponding features φ_i are thought to capture semantic concepts identified by the LLM internally while processing the input. The SAE is pre-trained on the LLM’s pre-training set (or equivalent) to discover $\varphi_1, \dots, \varphi_N$. Then, a separate automated feature explanation pipeline looks at all inputs where each specific feature φ_i is activated and endows the feature with a human-interpretable description (typically, by showing an LLM examples of inputs on which the feature was activated and asking it to conclude what it could represent [40-44]). The typical SAE encoding procedure is to multiply the input activation $x \in \mathbb{R}^d$ by the $d \times N$ matrix with columns $\varphi_1, \dots, \varphi_N$ followed by a non-linearity (e.g., ReLU) to obtain the non-zero coefficients α_i . Thus, the relevant features are those that maximize the inner product with x . They can be reported during user-LLM interaction together with their pre-computed descriptions, providing insight into how the LLM has processed the given prompt, as depicted in Fig. 2.

4.2.5. Privacy-preserving trend identification. We now illustrate at a high level how trends in user-LLM interactions can be identified from privately aggregated user interactions.

Let $x_j \in \mathbb{R}^d$ be the intermediate layer activation of user j . To simplify the discussion, suppose it has unit norm. The SAE produces an approximation $x_j \approx \sum_{i=1}^N \alpha_{ji} \varphi_i$, where only a few coefficients α_{ji} are non-zero.

Consider a particular feature φ_i , also of unit norm, say with the description “*cat or cat-related themes*”. If “cats” is a topic of user j ’s interaction with the LLM, then the corresponding coefficient α_{ji} would be large. Hence, the inner product $x_j^T \varphi_i$ would be large. (While $x_j^T \varphi_i$ is also impacted by the interference between φ_i and other SAE features, the SAE is trained to keep such interferences at a minimum.) At the population level, the prevalence of “cats” as a theme in a corpus of n user interactions can be quantified by the average inner product, $score(i) = \frac{1}{n} \sum_{j=1}^n x_j^T \varphi_i$. This is the KDE of φ_i under the inner-product kernel.

Next, we impose privacy. let $X = \frac{1}{n} \sum_{j=1}^n x_j$ be the average of all user activations $x_1 \dots x_n$. It can be made ϵ -DP as $\tilde{X} = X + v$, where v is a random noise vector with entries of magnitude proportional to $d/\epsilon n$. The exact population-level score of φ_i is equal to $score(i) = X^T \varphi_i$ by inner product linearity. However, it does not preserve privacy. Instead, we can report an ϵ -DP score as $\widetilde{score}(i) = \tilde{X}^T \varphi_i$. It is ϵ -DP by post-processing, as $\tilde{X}$ is already ϵ -DP. The noise v added to $\tilde{X}$ is chosen to obscure the contribution of any individual activation to $\widetilde{score}(i)$, thus it does not reveal whether any particular user has interacted with the LLM about cats. At the same time, if many users exhibited this feature in their LLM interaction, their accumulated contribution to $\widetilde{score}(i)$ overwhelms the noise and keeps the score noticeably large. Thus, by first privately aggregating the average user activation $\tilde{X}$, then computing $\widetilde{score}(i)$ for all SAE features φ_i and reporting the top scoring ones, population-level trends are identified.

The above description is, of course, a vast oversimplification. It disregards the actual numerical characteristics of intermediate LLM layer activations and of SAE features, and the distributed privacy requirement in the intended application to on-device LLM interactions. These issues can be addressed by the shuffled DP KDE framework, likely necessitating different kernels than the inner-product kernel, as explained in more detail below. In [1], this has been demonstrated by applying a topical identification scheme akin to the above over

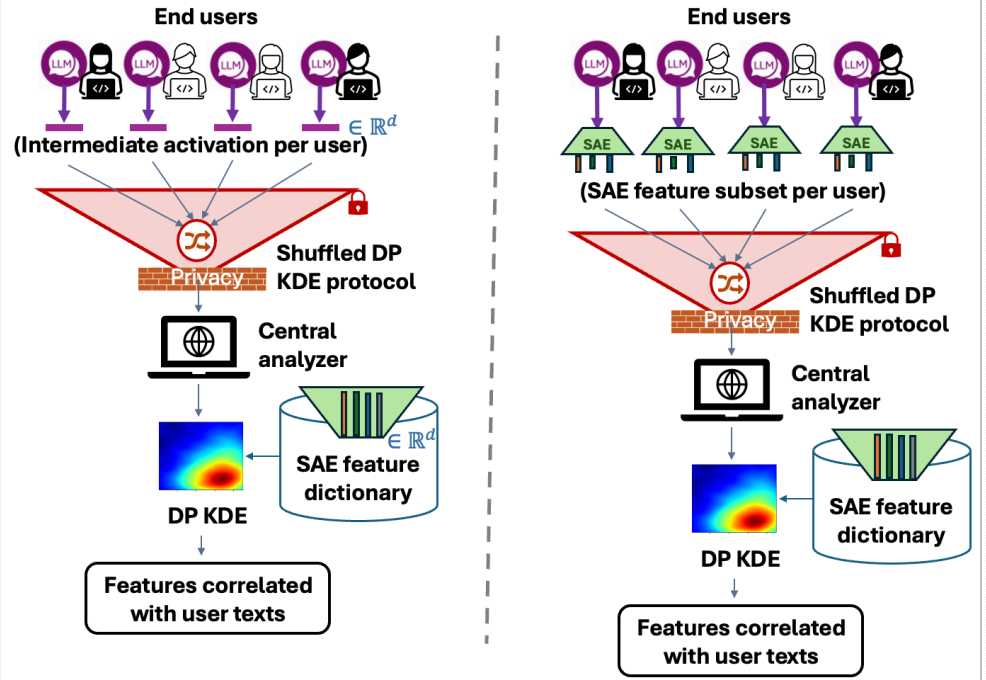


Figure 4: Two variants of the proposed approach. **Left: Server-side SAE.** Users share intermediate activation vectors with the central server. SAE is not run on end devices. **Right: Client-side SAE.** Users run the SAE on their end device and share sparse feature subsets with the central server.

transformer-based embeddings of user texts and keywords (cf. sec. 4.2.1). Since intermediate LLM layer activations are closely related to transformer-based embedding models, and since MI-SAEs are renowned for their effectiveness in interpreting intermediate LLM layer activations, the overall approach is expected to work in the proposed context too.

4.2.6. Algorithmic considerations. At a high level, as just outlined, MI-SAEs fit naturally into our proposed framework. Since the inner product similarity of a feature ϕ_i with an activation vector x is high when the feature is relevant to the input, the average similarity – i.e., the KDE, with a kernel equivalent to or correlated with the inner product – will be high on a corpus of intermediate activations collected from users whose interactions with the LLM exhibit that feature. The analyzer can compute a shuffled DP representation of the KDE function using the protocol from [1], then query it with each feature ϕ_i , and report the feature descriptions with highest DP KDE as capturing semantic themes of LLM interactions across the end user population.

However, implementing this high-level plan entails several intertwined design choices and challenges:

Server-side vs. client-side SAEs: The MI-SAE can be invoked either on the server or the client side. In the former case, clients never need to invoke the SAEs on their end devices. Instead, they directly share the intermediate layer activations of their on-device LLM interaction with the central analyzer via a shuffled DP protocol. The protocol aggregates those activations, and the analyzer subsequently queries them for SAE features (using DP KDE). This setup is depicted in Fig. 4-left. The other option, depicted in Fig. 4-right, is for the clients to run the SAE on their end-devices, and use a shuffled DP protocol to share with the analyzer an aggregation of the already extracted features. The pros and cons of each setup are discussed in the next section, 4.2.7.

Feature aggregation across tokens and (possibly) layers: MI-SAEs output the top features not per input, but per token, since each token induces its own intermediate activation vector. Key features of the input are most often activated on one or few tokens, but not all of them. This necessitates aggregating either the intermediate activations (in the server-side SAE setup) or the extracted features (in the client-side SAE setup) across the tokens of the input. Another possible

axis of aggregation is across layers of the model. MI-SAEs are trained per layer, and each layer SAE has its own separate set of features (for example, the Gemma-2-2B model used in Fig. 2 has a publicly released SAE for each of its 26 layers). It is common and mostly considered sufficient to focus on a single layer, though there may also be benefit in aggregating interpretability signals from multiple layers.

Feature interdependence: To draw insights from user-LLM interactions, we may want to test for more than one feature at a time. For example, consider again the example in Fig. 2. Suppose that by computing the DP KDE score of each feature, the analyzer finds that both Feature 15252 (“cat related themes”) and Feature 2194 (“Broadway musicals”) are prevalent themes in our corpus of user-LLM interactions. Does this necessarily mean that the musical “Cats” is a prevalent theme? The analyzer may now want to test whether felines and Broadway are two separate themes in the corpus, occurring irrespective of each other, or whether they often occur in the same user-LLM interaction, which would allude more specifically to the musical “Cats”. We therefore may want a system setup that allows the analyzer to test subsets of features rather than just single features.

These intertwined challenges can be summarized by the following overall challenge: *devise methods for feature extraction and aggregation per input, along with appropriate kernels for them, that on the one hand enable capturing rich semantic trends and on the other hand admit efficient shuffled DP KDE protocols.*

4.2.7 Framework instantiations and choices of kernels. We outline instantiations of the proposed framework for both the server-side SAE and client-SAE variant.

Notation: To ground the discussion, we introduce some notation. Let Φ be the set of SAE features. Let $\mathcal{X}$ be the domain of user inputs to the shuffled DP KDE protocol. Let $\mathcal{Y}$ be the domain of KDE queries. Let $k: \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$ be the chosen kernel, which measures the similarity between a user input and a query. The KDE function over a multiset X of user inputs from $\mathcal{X}$ is denoted $KDE_X: \mathcal{Y} \rightarrow \mathbb{R}$ and defined as $KDE_X(y) = \frac{1}{|X|} \sum_{x \in X} k(x, y)$. The shuffled DP KDE protocol returns a function description $\tilde{K}: \mathcal{Y} \rightarrow \mathbb{R}$ which is differentially private with respect to X , and such that the supremum root mean squared error (RMSE), $\sup_{y \in \mathcal{Y}} \sqrt{\mathbb{E} \left[\left(\tilde{K}(y) - KDE_X(y) \right)^2 \right]}$, is bounded as a function of the privacy parameters ϵ, δ and of the chosen kernel, and tends to 0 as the number of users in the corpus X grows (the expectation is over the internal randomness of the shuffled DP protocol). See [1] for details and exact bounds. Note that since the function description of $\tilde{K}$ itself is already differentially private, it can be evaluated on an unbounded number of queries without additional privacy leakage, by the DP post-processing property.

Server-side SAE setup: In the server-side SAE variant of the system, depicted in Fig. 4-left, end users use the shuffled DP KDE protocol to share their intermediate layer activations, without invoking the SAE on their end. Thus, the domain of user inputs is $\mathcal{X} = \mathbb{R}^d$, where d is the model intermediate activation dimension. The server-side KDE queries are the pre-trained SAE individual features, which are also vectors in $\mathbb{R}^d$, thus $\mathcal{Y} = \mathbb{R}^d$. Without distributed privacy considerations, the straightforward choice of kernel is the inner product, $k(x, y) = x^T y$, since as explained earlier (sec. 4.2.4), this is what the SAE is trained for. However, experimental findings in [1] indicate that in the shuffled DP model, the gaussian kernel $k(x, y) = \exp(-\|x - y\|^2)$ (equivalent to the exponent of the inner product if x, y are normalized) may lead to better privacy-utility-efficiency trade-offs for vectors generated by models trained for inner product similarity. Both kernels admit efficient shuffled DP KDE protocols in the framework from [1].

The server-side SAE variant has the clear computational advantage of not requiring users to run the SAE, thus adding almost no additional computational burden to the end devices (the only additional computation required of them is

participation in the shuffled DP KDE protocol, which is lightweight). Intermediate activations need to be aggregated on the user device. However, this setup supports querying individual features but not subsets of features.

Client-side SAE setup: In the client-side SAE variant of the system, each user locally runs the MI-SAE on the end device to extract SAE features and uses the shuffled DP protocol to share with the server the already extracted features. While this may add considerable computational burden to the end devices (see discussion of computational efficiency in the next section 4.3), it may lead to more refined interpretability insights, and in particular ones that preserve feature interdependence, as features are extracted per user prior to imposing the privacy barrier (cf. Fig 4-right).

How can extracted features be aggregated with shuffled DP? One natural option is for the analyzer to compute a privacy-preserving histogram of extracted features (there are indeed available shuffled DP histogram protocols).

However, this fails to preserve feature interdependence. Histograms over subsets of features are technically possible, albeit each subset size requires a separate histogram, and the privacy budget would need to be split among them.

A more intriguing possibility is the *Jaccard kernel*. In our notation, each user input to the shuffled DP protocol is a subset of features, thus $\mathcal{X} = 2^\Phi$ (recall that Φ is the set of the pre-trained SAE features); and each server query is also a subset of features, thus $\mathcal{Y} = 2^\Phi$. The Jaccard kernel between two subsets of features is the ratio between their intersection and union sizes, namely $k(x, y) = \frac{|x \cap y|}{|x \cup y|}$ for $x \in \mathcal{X}$ and $y \in \mathcal{Y}$. The classical MinHash algorithm [28]

forms an efficient hashing-based algorithmic technique for the Jaccard kernel, which renders it compatible with the PI's DP KDE method in [1-2]. Thus, an efficient shuffled DP KDE protocol for the Jaccard kernel is possible. Using the Jaccard DP KDE, the analyzer can first query individual features as singleton subsets (say, {"cats"} and {"Broadway"}) as two separate KDE queries) and then proceed to query non-singleton subsets (say, {"cats", "Broadway"}) as one KDE query) with the same DP KDE function to gain insight into feature interdependence.

The client-side SAE setup, as discussed so far, assumes that each client extracts and shares with the server an unweighted subset of features. However, as described in subsection 4.2.4, the SAE also returns a coefficient α_i for each feature ϕ_i it extracts; the higher α_i is, the more prominent the feature ϕ_i is in the input. The coefficients can be used in several ways. Simpler options include the client using the top- m features according to the coefficients for some fixed m , or all features whose coefficients are above a fixed threshold. Another option is to use weighted variants of the Jaccard kernel and of the MinHash technique. There are several such variants available in the literature [29-31].

Finally, it may also be beneficial to explore hybrid variants of the server-side SAE and client-side SAE setups.

4.3. Computational Efficiency Considerations

The proposed framework presents certain computational efficiency challenges that must be addressed. The main issue is the computational cost associated with the large number of SAE features. For MI-SAEs to achieve interpretable feature decompositions with the requisite sparsity, their total number of features $N = |\Phi|$ needs to be large. For example, the pre-trained SAE for Gemma-2-2B used in the examples so far has $N = 16,384$ features in total, each a vector in $\mathbb{R}^d$ with $d = 1,024$. Extracting features from a given intermediate layer activation involves multiplying the $N \times d$ matrix of SAE features by the d -dimensional activation vector. This needs to be done for each token.

In the client-side SAE setup, exact feature extraction may impose a prohibitively heavy computational burden on the user end device. A possible mitigation is to invoke approximate methods. Notice that feature extraction amounts to a maximum inner product (MIP) search where the intermediate activation vector serves as the query and the collection of SAE feature vectors serves as the database. Fast approximate MIP is a well-researched problem with numerous available solutions that can be used in the context of our proposed system [32-33].

In both the server-side SAE and client-side SAE setups, the server is also expected to enumerate over all features and query the DP KDE with each. While enumerating over (say) $N = 16,384$ features may be computationally taxing for an end device, it should not pose an issue for the central server. Evaluating the DP KDE with the protocol of [1] generally requires minimal computation over the features and involves vectorized operations that can be parallelized across features, and much of it can be done as preprocessing in advance (before collecting user data). Therefore, the proposed system is expected to be computationally viable.

4.4. Methodology, Feasibility and Implementation

The proposed research involves a prototype implementation of the system. We overview the available assets and resources. The shuffled DP KDE protocols from [1] have a publicly available python implementation released by the PI as part of that work. For LLM and SAE components, there are several open-weight models – including few variants of GPT2 and Gemma – that also have publicly released pre-trained SAEs with accompanying feature explanations, with a python API to extract intermediate layer activations and SAE features. The CS&AI department in Tel Aviv university has GPU cluster with a GPU reserved for the PI and several more available on a shared basis. Thus, all the requisite assets for implementing a prototype are available, and it can be built within the proposal's time frame with the resources available in the academic unit where the research is to be performed.

5. Significance, innovation and potential benefits of the proposed research

5.1 Scientific Innovation

This research introduces several novel contributions to the intersection of privacy-preserving machine learning and AI interpretability:

Methodological innovation: The integration of SAE-based mechanistic interpretability with shuffled differential privacy represents a fundamentally new approach to understanding AI system behavior in decentralized settings while preserving privacy. This methodology bridges the gap between the need to monitor and analyze on-device usage and the imperative for privacy protection.

Algorithmic contributions: Our proposed application of DP KDE to SAE features introduces entails developing novel algorithmic tools for aggregation and similarity measurement of sparse feature activations of user-LLM interactions. The algorithm design specifically accounts for the unique properties of SAE feature spaces, including sparsity patterns and semantic structure.

5.2 Potential Benefits

The ability to draw population-level patterns and insights from on-device user-LLM interactions under individual privacy preservation provides significant benefits and opportunities.

User experience optimization: Understanding how and for what ends users engage with AI systems on end devices may promote benefits like improved interfaces, better response quality and relevance, more intuitive AI behaviors, and accelerated development and improvement of on-device AI agents that successfully meet users' goals and needs.

Industry applications: The proposed method enables technology companies to better understand their AI systems' usage and behavior “in the wild”, on end user devices, while complying with privacy regulations and maintaining user trust. This capability is increasingly important as AI systems become more prevalent in consumer applications.

AI safety and alignment: The ability to analyze patterns of on-device user-LLM interactions may provide insight into potential failure modes, alignment issues, and emergent behaviors. Visibility into these matters is key to ensuring the safe deployment of increasingly powerful AI systems.

5.3 Broader Impact

Privacy rights and technology: The proposed research contributes to the broader goal of developing technologies that respect human privacy rights while enabling beneficial analysis and improvement of AI systems. The framework provides a concrete example of privacy-by-design principles in AI research.

Policy and governance: The proposed research provides empirical evidence for the feasibility and effectiveness of privacy-preserving AI analysis, which may aid in informing policy discussions about AI governance, privacy regulation, and the balance between innovation and protection. As AI technology increasingly permeates everyday life, and policy discussion and formulation around its privacy implications are ongoing, concrete demonstration for the feasibility of privacy-preserving analysis may lead to more grounded discussion and better aligned policies.

6. Applicability

6.1 Data Collection for Field Studies in AI research

Applicability in AI research: The proposed framework directly supports researchers studying LLM behavior, human-AI interaction, and AI safety by providing tools for large-scale analysis while maintaining privacy protections. This enables the collection and curation of LLM usage data in field studies at a scope otherwise impeded by privacy constraints. The data and insights obtained from such field studies could open new research directions in AI behavior and human-AI interaction in real-world settings.

6.2 Industry and Commercial Applications

Technology companies offering LLM services can use the proposed method to understand user interaction patterns, identify improvement opportunities, and ensure system reliability while maintaining user privacy and regulatory compliance. Moreover, the privacy-preserving nature of the proposed framework makes it applicable to sensitive domains such as healthcare, finance, and legal services, where AI analysis must comply with strict privacy regulations while providing valuable insights.

6.3 Technical Transferability

Other AI architectures: While focused on LLMs, the core principles of the proposed approach are grounded in broader scientific fundamentals of density estimation, privacy and interpretability. As such, it can be adapted to other neural network architectures and AI systems, expanding its potential applicability across the broader AI landscape, and ensuring its continued relevance as the field evolves.

Other system architectures: While focused on decentralized on-device user-LLM interaction, the proposed framework is applicable to other AI deployment scenarios. For example, it can be applied to centralized LLM services too. Even when users interact with a remotely hosted LLM through a web interface, and centralized curation of user interactions is technically possible, it may be prohibitive of undesirable due to regulatory constraints or increased susceptibility to attacks. In such cases, distributed privacy-preserving curation may be preferred, which our proposed method can fulfill.

7. Work plan and Gantt

#	Task	6	12	18	24	30	36
1	Algorithmic design	x	x	x	x		
2	Data curation	x	x				
3	Initial experimental setup	x	x	x			
4	Theoretical study	x	x	x	x		
5	Prototype development			x	x	x	x
6	Comprehensive empirical results				x	x	x

8. Bibliography

- [1] Tal Wagner. "Learning from End User Data with Shuffled Differential Privacy over Kernel Densities." In International Conference on Learning Representations (ICLR) 2025.
- [2] Tal Wagner, Yonatan Naamad, and Nina Mishra. "Fast private kernel density estimation via locality sensitive quantization". In International Conference on Machine Learning (ICML), 2023.
- * [3] Arturs Backurs, Zinan Lin, Sepideh Mahabadi, Sandeep Silwal, and Jakub Tarnawski. "Efficiently computing similarities to private datasets". In International Conference on Learning Representations (ICLR), 2024.
- * [4] Alexandr Andoni, Piotr Indyk, Sepideh Mahabadi, and Shyam Narayanan. "Differentially private approximate near neighbor counting in high dimensions." Advances in Neural Information Processing Systems (NeurIPS), 2023.
- [5] Benjamin Coleman and Anshumali Shrivastava. "A one-pass distributed and private sketch for kernel sums with applications to machine learning at scale." In Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 2021.
- * [6] Francesco Alda and Benjamin IP Rubinstein. "The Bernstein mechanism: Function release under differential privacy". In Thirty-First AAAI Conference on Artificial Intelligence, 2017.
- * [7] Daogao Liu, Edith Cohen, Badih Ghazi, Peter Kairouz, Pritish Kamath, Alexander Knop, Ravi Kumar, Pasin Manurangsi, Adam Sealfon, Da Yu, and Chiyuan Zhang. "Urania: Differentially Private Insights into AI Use." arXiv preprint arXiv:2506.04681 (2025).
- [8] Alex Tamkin, Miles McCain, Kunal Handa, Esin Durmus, Liane Lovitt, Ankur Rathi, Saffron Huang, Alfred Mountfield, Jerry Hong, Stuart Ritchie, Michael Stern, Brian Clarke, Landon Goldberg, Theodore R. Sumers, Jared Mueller, William McEachen, Wes Mitchell, Shan Carter, Jack Clark, Jared Kaplan, Deep Ganguli. "Clio: Privacy-Preserving Insights into Real-World AI Use." arXiv preprint arXiv:2412.13678 (2024).
- [9] Cynthia Dwork, and Aaron Roth. "The algorithmic foundations of differential privacy." Foundations and Trends® in Theoretical Computer Science 9, no. 3–4 (2014): 211-407.
- [10] Natalia Ponomareva, Hussein Hazimeh, Alex Kurakin, Zheng Xu, Carson Denison, H. Brendan McMahan, Sergei Vassilvitskii, Steve Chien, and Abhradeep Guha Thakurta. "How to dp-fy ml: A practical guide to machine learning with differential privacy." Journal of Artificial Intelligence Research 77 (2023): 1113-1201.
- [11] Andrea Bittau, Úlfar Erlingsson, Petros Maniatis, Ilya Mironov, Ananth Raghunathan, David Lie, Mitch Rudominer, Ushasree Kode, Julien Tinnes, and Bernhard Seefeld. "Prochlo: Strong privacy for analytics in the crowd." In Proceedings of the 26th symposium on operating systems principles, pp. 441-459. 2017.
- * [12] Úlfar Erlingsson, Vitaly Feldman, Ilya Mironov, Ananth Raghunathan, Kunal Talwar, and Abhradeep Thakurta. "Amplification by shuffling: From local to central differential privacy via anonymity." In Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 2019.
- [13] Albert Cheu, Adam Smith, Jonathan Ullman, David Zeber, and Maxim Zhilyaev. "Distributed differential privacy via shuffling." In Advances in Cryptology–EUROCRYPT 2019: 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, 2019.
- [14] Borja Balle, James Bell, Adria Gascon, and Kobbi Nissim. "The privacy blanket of the shuffle model." In Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, 2019.
- [15] Borja Balle, Peter Kairouz, Brendan McMahan, Om Thakkar, and Abhradeep Guha Thakurta. "Privacy amplification via random check-ins". Advances in Neural Information Processing Systems (NeurIPS), 2020.

- [16] Antonious M Girgis, Deepesh Data, Suhas Diggavi, Ananda Theertha Suresh, and Peter Kairouz. "On the Renyi differential privacy of the shuffle model". In Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 2021.
- [17] Vitaly Feldman, Audra McMillan, and Kunal Talwar. "Hiding among the clones: A simple and nearly optimal analysis of privacy amplification by shuffling". In 2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS), IEEE, 2022.
- [18] Vitaly Feldman, Audra McMillan, and Kunal Talwar. "Stronger privacy amplification by shuffling for Renyi and approximate differential privacy". In Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 2023.
- [19] Mingxun Zhou and Elaine Shi. The power of the differentially oblivious shuffle in distributed privacy mechanisms. Cryptology ePrint Archive, 2022.
- [20] Ghazi Badih, Ravi Kumar, Pasin Manurangsi, and Rasmus Pagh. "Private counting from anonymous messages: Near-optimal accuracy with vanishing communication overhead". In International Conference on Machine Learning (ICML), 2020.
- [21] Badih Ghazi, Noah Golowich, Ravi Kumar, Pasin Manurangsi, Rasmus Pagh, and Ameya Velingker. "Pure differentially private summation from anonymous messages". arXiv preprint arXiv:2002.01919, 2020.
- [22] Albert Cheu and Chao Yan. "Pure differential privacy from secure intermediaries". arXiv preprint arXiv:2112.10032, 2021.
- [23] Albert Cheu, Matthew Joseph, Jieming Mao, and Binghui Peng. "Shuffle private stochastic convex optimization." arXiv preprint arXiv:2106.09805 (2021).
- [24] Antonious Girgis, Deepesh Data, Suhas Diggavi, Peter Kairouz, and Ananda Theertha Suresh. "Shuffled model of differential privacy in federated learning." In International Conference on Artificial Intelligence and Statistics (AISTATS), 2021.
- [25] Xingyu Zhou and Sayak Ray Chowdhury. "On differentially private federated linear contextual bandits". arXiv preprint 2302.13945, 2023.
- [26] Sayak Ray Chowdhury and Xingyu Zhou. "Shuffle private linear contextual bandits". arXiv preprint 2202.05567, 2022.
- [27] Jay Tenenbaum, Haim Kaplan, Yishay Mansour, and Uri Stemmer. "Differentially private multiarmed bandits in the shuffle model". Advances in Neural Information Processing Systems (NeurIPS), 2021.
- [28] Andrei Z. Broder. "On the resemblance and containment of documents." In Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No. 97TB100171), pp. 21-29. IEEE, 1997.
- [29] Ondrej Chum, James Philbin, and Andrew Zisserman. "Near duplicate image detection: Min-hash and TF-IDF weighting." In BMVC, vol. 810, pp. 812-815. 2008.
- [30] Ioffe, Sergey. "Improved consistent sampling, weighted minhash and l1 sketching." In 2010 IEEE international conference on data mining, pp. 246-255. IEEE, 2010.
- [31] Anshumali Shrivastava. "Exact weighted minwise hashing in constant time." arXiv preprint 1602.08393 (2016).
- [32] Anshumali Shrivastava and Ping Li. "Asymmetric LSH (ALSH) for sublinear time maximum inner product search (MIPS)." Advances in neural information processing systems (NeurIPS), 2014.

- [33] Behnam Neyshabur, and Nathan Srebro. "On symmetric and asymmetric lshs for inner product search." In International Conference on Machine Learning (ICML), 2015.
- [34] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. "Sparse autoencoders find highly interpretable features in language models". arXiv preprint arXiv:2309.08600 (2023).
- [35] Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. "Scaling and evaluating sparse autoencoders". arXiv preprint arXiv:2406.04093 (2024).
- [36] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, Alex Tamkin, Esin Durmus, Tristan Hume, Francesco Mosconi, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, Tom Henighan. "Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet". 2024. <https://transformer-circuits.pub/2024/scaling-monosemanticity/>
- [37] Tom Lieberum, Senthoooran Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda. "Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2." arXiv preprint arXiv:2408.05147 (2024).
- [38] Joseph I. Bloom, "Open Source Sparse Autoencoders for all Residual Stream Layers of GPT2-Small". 2024. <https://www.alignmentforum.org/posts/f9EgflSurAiqRJySD/open-source-sparse-autoencoders-for-all-residual-stream>
- [39] Neuronpedia, <https://www.neuronpedia.org/>
- [40] Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeff Wu, and William Saunders. "Language models can explain neurons in language models". 2023. <https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html>
- [41] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nicholas L Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, Chris Olah. "Towards monosemanticity: Decomposing language models with dictionary learning". 2023. URL: <https://transformer-circuits.pub/2023/monosemantic-features/index.html>
- [42] Gonçalo Paulo, Alex Mallen, Caden Juang, and Nora Belrose. "Automatically interpreting millions of features in large language models". Preprint, arXiv:2410.13928, 2024.
- [43] Dami Choi, Vincent Huang, Kevin Meng, Daniel D Johnson, Jacob Steinhardt, and Sarah Schwettmann. "Scaling automatic neuron description". 2024. <https://transluce.org/neuron-descriptions>
- [44] Gur-Arieh, Yoav, Roy Mayan, Chen Agassy, Atticus Geiger, and Mor Geva. "Enhancing Automated Interpretability with Output-Centric Feature Descriptions." arXiv preprint arXiv:2501.08319 (2025).
- [45] Guanqiao Qu, Qiyuan Chen, Wei Wei, Zheng Lin, Xianhao Chen, and Kaibin Huang. "Mobile edge intelligence for large language models: A contemporary survey." IEEE Communications Surveys & Tutorials (2025).
- [46] Daliang Xu, Wangsong Yin, Hao Zhang, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. "EdgeLLM: Fast On-device LLM Inference with Speculative Decoding." IEEE Trans. on Mobile Computing (2024).
- [47] Xinyi Hou, Jiahao Han, Yanjie Zhao, and Haoyu Wang. "Unveiling the Landscape of LLM Deployment in the Wild: An Empirical Study." arXiv preprint arXiv:2505.02502 (2025).